Azure · Python · Microsoft Agent Framework

Harness Engineering

Seven patterns for putting an AI agent into production — re-built in Python on managed Azure primitives.

A local-first practice project · all 7 lessons implemented & tested

First: which "harness"?

The term is overloaded. Two different things share the name — this project is the first one.

This project

Runtime harness

The scaffolding wrapped **around a deployed agent**: guardrails, memory, tools, orchestration, human-in-the-loop. Built here in **Python on the Microsoft Agent Framework**.

Not this project

Dev-time harness

OpenAI's "humans steer, agents execute" — the environment & feedback loops that let **coding agents write your software**. Same words, different layer.

We're building the harness an agent runs *inside*, not the harness a team builds software *with*. **From here on we'll call it the runtime** — same thing, no collision.

The thesis

Seven runtime patterns, built in **pure Python** on the **Microsoft Agent Framework**:

- **Agent Framework gives you the harness** — middleware, checkpointed workflows, multi-agent orchestration, human-in-the-loop.
- **Azure AI Foundry gives you the agents** — the model call, the tool loop, managed threads, sandboxed tools.
- You write the **orchestration, the guardrails and the observability** — not the plumbing underneath them.

Working rule throughout: `keep implementations thin`. If a pattern can be solved by configuring a managed primitive, don't hand-roll it.

An operating system for your agent

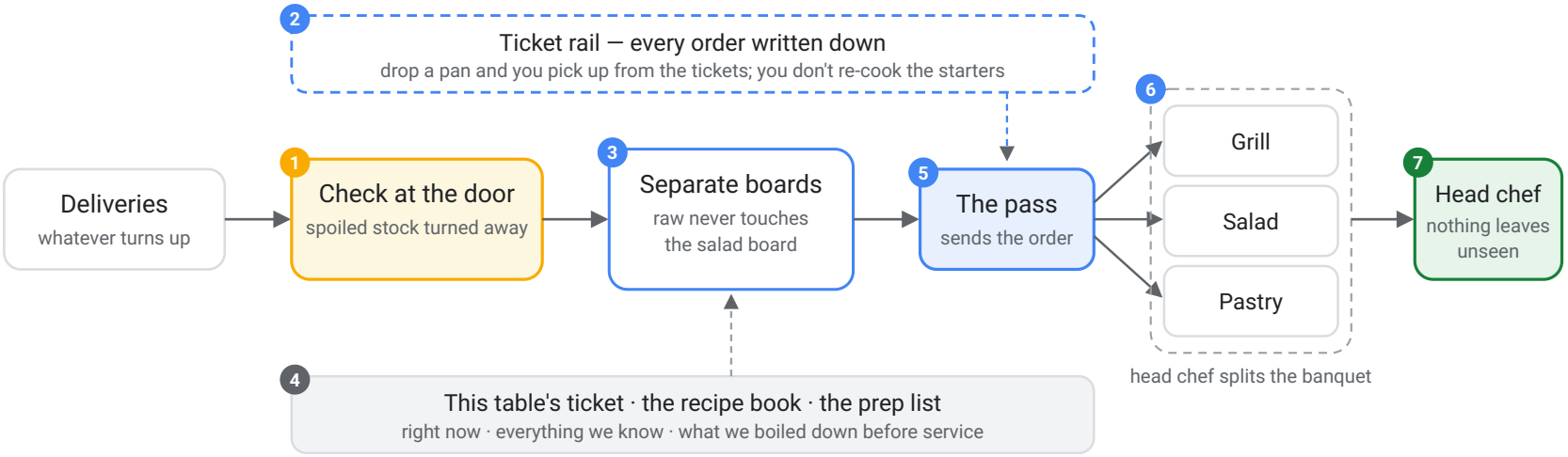
If the model is a **process**, the runtime is the **OS**.

#	PATTERN	WHAT IT IS, IN OS TERMS
1	Guardrails	syscall validation — `seccomp`
2	Durable execution	journaling · checkpoint / restore
3	Sandboxing	process isolation · namespaces
4	Memory	RAM · disk · paging
5	Orchestration	the scheduler
6	Supervision	a process supervision tree
7	Human-in-the-loop	`sudo`

Nobody writes their own scheduler any more. **Azure gives you the OS; you write the program.** One honest gap: ring 0 is enforced by *hardware* — this boundary is software. `state the leak`.

Or: the same seven, without the jargon

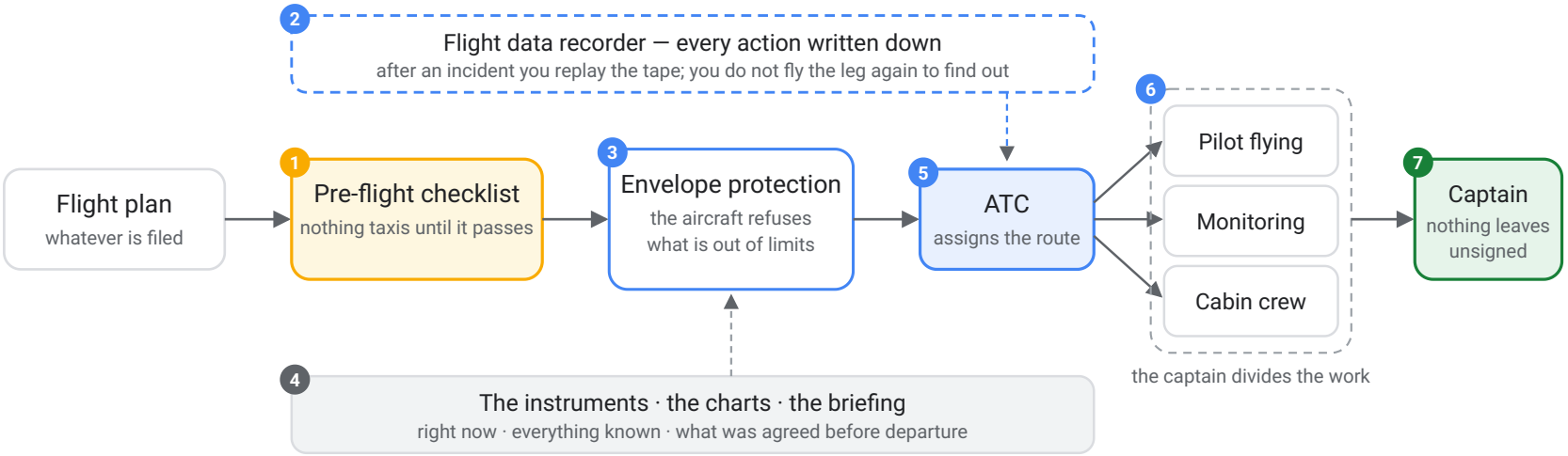
If the model is the **cook**, the runtime is the **kitchen around them**.



Nobody builds a kitchen from scratch to open a restaurant. **Azure gives you the kitchen; you decide the menu.**

Or: the same seven, on a flight deck

If the model is the **autopilot**, the runtime is the **aircraft around it**.



Nobody builds an airframe to run an airline. **Azure gives you the aircraft; you fly the route.** The honest gap: aviation earned this from *decades of mandatory accident reporting*. We have no such corpus — `state the leak`.

Two layers do the work

Azure provides — the agents

AI Foundry · Agent Service. Model calls + the tool loop, managed **threads** (memory), and sandboxed tools: Code Interpreter, File Search, Function, AI Search.

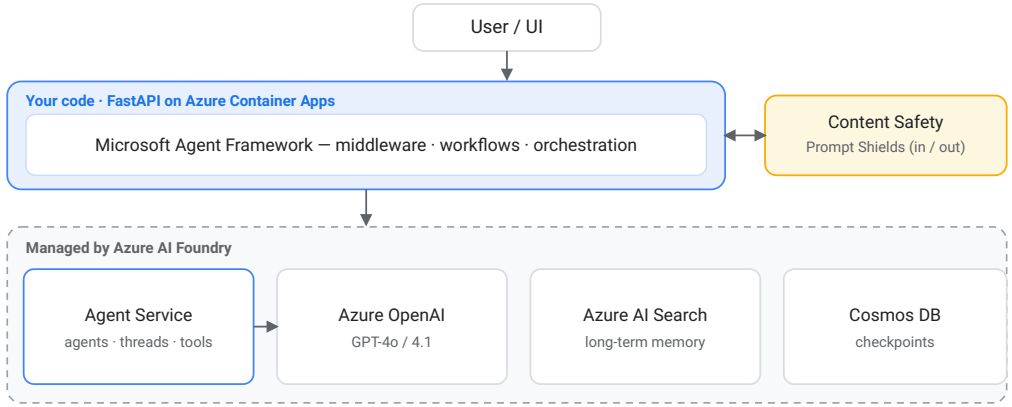
You build — the harness around them

Microsoft Agent Framework. Middleware/guardrails, durable checkpointed workflows, multi-agent orchestration, and human-in-the-loop.

Our own code is a thin **FastAPI orchestration API** (destined for Azure Container Apps) that drives these agents.

`Agent Framework is public preview — pin versions`

Architecture



You host only the API; everything in the dashed band is managed by Azure. Cross-cutting: App Insights (OpenTelemetry traces) · Entra Managed Identity (keyless, `DefaultAzureCredential`).

The one convention: the seam pattern

Every Azure dependency is stubbed behind a seam, so the whole suite runs with **no cloud and no network**.

- A `typing.Protocol` (or injectable callable/table) = the swap point.
- A **local implementation** whose docstring **names the Azure service** it stands in for.
- It honestly documents where the local stand-in is weaker — **"state the leak."**

```
# durable/checkpoint.py — same interface, local SQLite today, Cosmos DB later
class CheckpointStore(Protocol):
    def load(self, run_id: str) -> Checkpoint | None: ...
    def save(self, checkpoint: Checkpoint) -> None: ...
```

Swap local → Azure without touching the workflow logic. **Match this pattern when adding code.**

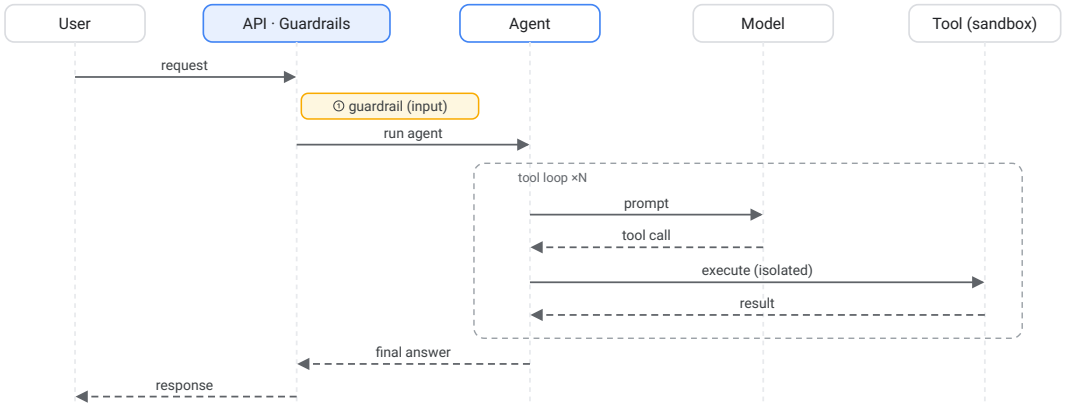
The seven lessons

#	PATTERN	YOU BUILD (LOCAL MODULE)	
1	Agent Harness	`guardrails.py` + `app.py`	local ✓
2	Durable Execution	`workflow.py` + `durable/`	local ✓
3	Secure Sandboxing	`sandbox.py`	local ✓
4	Advanced Memory	`memory/` (thread·knowledge·summarize)	local ✓
5	Orchestration	`orchestration/router.py`	local ✓
6	Hierarchical Supervision	`orchestration/supervisor.py`	local ✓
7	Human-in-the-Loop	`hitl.py` (on `durable/`)	local ✓

"local ✓" = runnable + unit-tested with Azure stubbed behind a seam. Azure-wired impls are the next increment.

L1 Agent Harness

Guardrails wrap the model as middleware — with a tool loop in the middle.



- `guardrails.check_input` blocks empty, oversize & injection markers before the model — a **Content Safety Prompt Shields** stand-in.
- Only `/health` and `/chat` are wired today; the other six lessons are standalone modules.

```
# app.py — the guardrail runs BEFORE the model, as middleware
verdict = check_input(user_message)
if not verdict.allowed:
    return Blocked(verdict.reason)      # the model is never called
```

L1+ Evasion-resistant safety layer

A naive marker list is trivially bypassed. `safety/` hardens it into a real Prompt-Shields stand-in.

`normalize`

Folds obfuscation — homoglyphs, zero-width chars, spacing — before matching.

`detectors`

Heuristic **attack-family** detectors run over the folded text.

`redteam`

A corpus of attacks + a scorer to measure the layer against itself.

state the leak

Still **heuristic, not ML** — the docstring says so. No false confidence.

Threat model: `docs/safety-layer.md`.

```
# safety/ — fold the disguise, THEN match
folded = normalize(msg)        # homoglyphs · zero-width · spacing
verdict = check_input(msg)     # markers re-checked over folded text
```

L1 Guardrails — the example

The same `check_input` the test suite exercises, on a disguised attack.

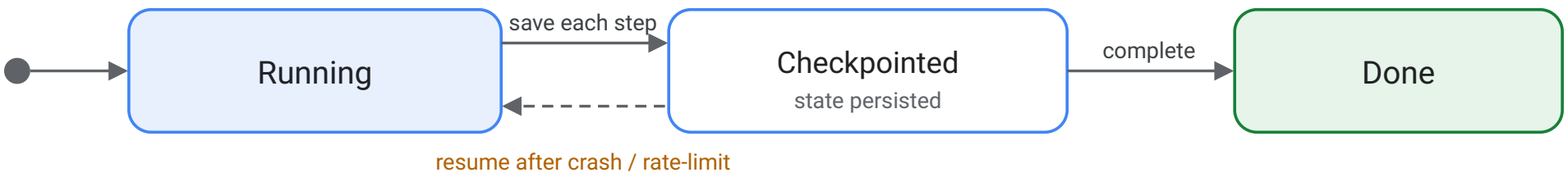
```
# the naive check most demos ship — a substring scan
naive = next((m for m in INJECTION_MARKERS if m in msg.lower()), None)
#   -> None. it sails straight through.

# look at what is actually in the string
#   pos  5   U+0435   CYRILLIC SMALL LETTER IE

folded  = normalize(msg)      # fold the disguise
verdict = check_input(msg)    # then match again
#   -> BLOCKED: possible prompt injection (obfuscated)
```

L2 Durable Execution

A run must **survive a crash** — resume from the last checkpoint instead of restarting.



- `workflow.py` runs steps; `durable/` persists a **Checkpoint(run_id, step, state)** after each.
- Local **SQLite** today → **Cosmos DB** later → **localStorage** in this browser. Same interface; `run_durably` never knows.

```
# workflow.py — checkpoint after every step; resume skips completed work
state = run_durably(run_id, steps, store)
```

L2 Durable execution — the example

A real crash, a real page reload, and a resume that skips completed work.

```
# same CheckpointStore Protocol as SQLite and Cosmos
store = BrowserCheckpointStore(js.localStorage)

run_durably("banquet-42", STEPS, store)
#   ran step-1 · ran step-2 · CRASH: rate limited

# --- reload the page. the interpreter and every object in it die ---

run_durably("banquet-42", STEPS, store)   # same run id
#   ran step-3 · ran step-4
#   step-1 and step-2 never ran again
```

L3 Secure Sandboxing

Agent-generated code can't run in your process. `sandbox.py` isolates it.

- Executes in an **isolated subprocess** with a hard **timeout**.
- Local stand-in for the Agent Service **Code Interpreter** tool.
- State the leak: a subprocess is *not* a real sandbox — the managed tool gives true isolation, network controls, and resource limits.

The pattern that matters: never `exec()` untrusted output in-process — put a boundary (and a clock) around it.

```
# sandbox.py — isolated subprocess, hard timeout
r = run_code(model_written_code, timeout=5.0)
r.ok, r.timed_out, r.stdout
```

L3 Sandboxing — the example

A subprocess with a clock — and a runtime that refuses outright.

```
r = run_code("print('hello from inside the sandbox')")
r.ok, r.exit_code, r.stdout

r = run_code("while True: pass", timeout=1.0)
r.timed_out          # True — contained by the clock

# but run the same call in this browser:
#   OSError — the runtime refuses to spawn a process at all
#   a real boundary refuses; a teaching stand-in only pretends
```

L4 Advanced Memory

Three memory shapes, three stand-ins — under `memory/`.

short-term
`thread`
Per-conversation history. Stands in for Agent Service **managed threads**.

long-term
`knowledge`
Retrievable facts. Stands in for **Azure AI Search** (RAG).

compaction
`summarize`
Condense history to fit the context window.

Short-term = the conversation; long-term = a knowledge store the agent can query; summarization = keep both affordable within the token budget.

```
# three memories, three lifetimes
threads.append(tid, "user", msg)      # the turn
kb.search("duplicate charge", k=2)    # the corpus
summarize_thread(history, max_chars=500)  # the context budget
```

L4 Memory — the example

Three memories, three lifetimes — running on real SQLite in the browser.

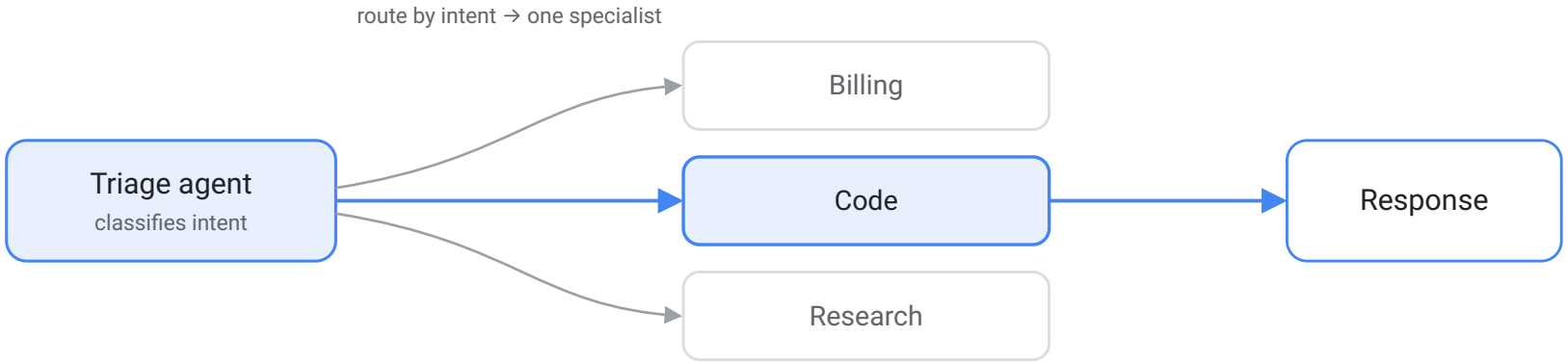
```
threads = SQLiteThreadStore(":memory:")
threads.append("t-1", "user", "my invoice is wrong")
threads.history("t-1")                      # the turn

kb = InMemoryKnowledgeStore()
kb.add("refunds", "Duplicate charges are refunded in five days.")
kb.search("duplicate charge refund", k=2)  # the corpus

summarize_thread(history, max_chars=120, keep=1)
#   user: my invoice is wrong
#   [...4 earlier turns omitted...]
```

L5 Orchestration — routing & handoff

A **triage agent** classifies intent and hands off to the right specialist.



- `orchestration/router.py` — intent → specialist, the Agent Framework **handoff** pattern.
- One entry point, many specialists; each specialist stays focused and small.

```
# orchestration/router.py — intent → specialist
Router().route("refund my invoice")
# RouteDecision(specialist='billing', matched_intent='invoice')
```

L5 Routing — the example

Three requests routed, one paraphrase missed — and the leak that causes it.

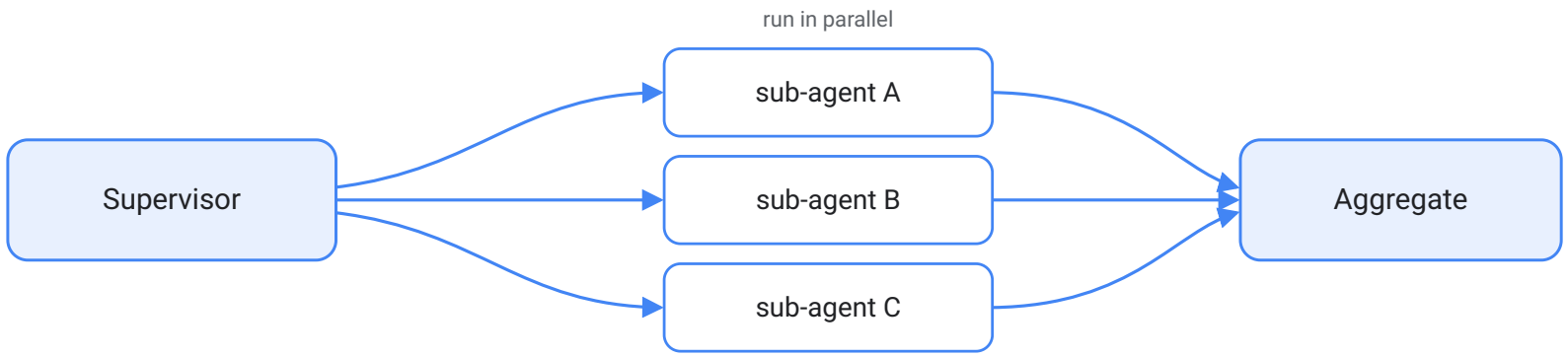
```
Router().route("I need a refund on my invoice")
#   RouteDecision(specialist='billing', matched_intent='invoice')

Router().route("the project won't build any more")
#   RouteDecision(specialist='fallback', matched_intent=None)
#   a human reads that as a CODE problem instantly.
#   'compile' is in the table; "won't build" is not.

Router({"won't build": Specialist.CODE}).route(...)
#   patched in one line — the treadmill a triage agent removes
```

L6 Hierarchical Supervision

Routing escalated: a **supervisor** decomposes a task, fans out, and aggregates.



- `orchestration/supervisor.py` — parallel **fan-out / fan-in**.
- Maps to Agent Framework **Magentic / concurrent** orchestration.

```
# orchestration/supervisor.py — fan-out / fan-in
agg = Supervisor(worker=sub_agent).run("a; b; c")
agg.succeeded, agg.failed   # one failure never sinks the batch
```

Routing picks *one* agent; supervision runs *many* in parallel and merges their results.

L6 Supervision — the example

Fan out, one sub-agent fails, and the batch survives in a stable order.

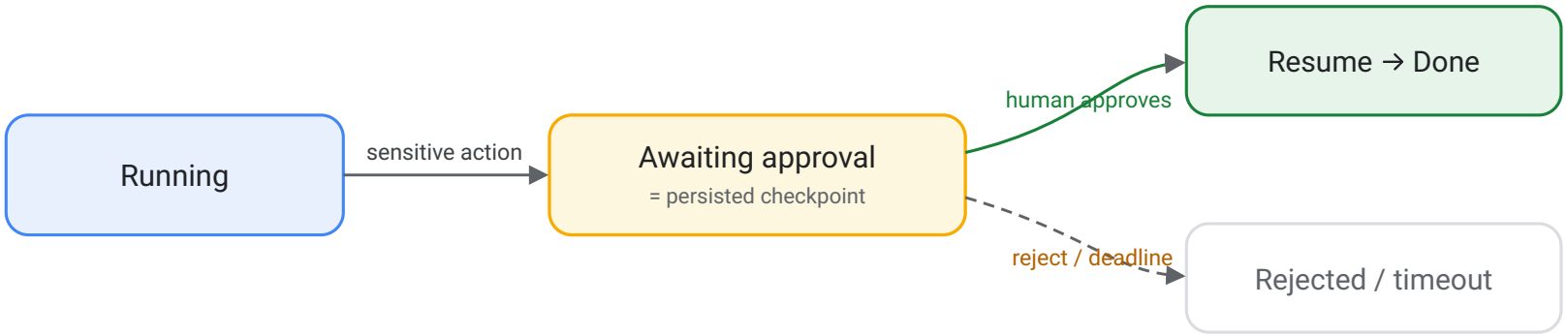
```
sup = Supervisor(worker=sub_agent, executor_factory=SyncExecutor)
agg = sup.run("summarise the outage; check pricing; draft the note")

for r in agg.results:      # decomposition order, not completion
    r.subtask, r.output, r.error

agg.succeeded, agg.failed  # 2, 1
#   one sub-agent raised; the exception never escaped run()
#   executor_factory is a seam: this browser has no threads,
#   so the isolation is real and the parallelism is not
```

L7 Human-in-the-Loop

Pause before a sensitive action, wait for a human, then resume — built on L2's checkpoints.



- `hitl.py` — checkpointed **suspend / resume** with a **deadline** (reject on timeout).
- Reuses the durable store: a paused run is just a persisted checkpoint waiting on input.

```
# hitl.py — suspend, then resume against a deadline
request_approval(run_id, prompt, store, now=t0, timeout_seconds=60)
resume(run_id, decision=True, store=store, now=t1, action=refund)
```

L7 Human-in-the-loop — the example

Approve, reject, and the one nobody expects — a yes that arrives too late.

```
request_approval(run_id, "Refund 40.00 GBP", store,
                 now=0.0, timeout_seconds=60.0)
#   status=awaiting_approval   performed=False

resume(run_id, True,  store, now=10.0,   action=issue_refund)
#   approved — action performed
resume(run_id, False, store, now=10.0,   action=issue_refund)
#   rejected — action NOT performed
resume(run_id, True,  store, now=7200.0, action=issue_refund)
#   timed_out — the deadline is checked BEFORE the decision
```

The docs are part of the test suite

Knowledge lives in the repo and is **mechanically validated** — stale docs turn the suite red.

- `tests/test_docs.py` checks every `.md` / `.excalidraw`: balanced fences, resolvable anchors, valid Excalidraw JSON, no HTML-escape artifacts.
- Heavier Mermaid render is opt-in: `HARNESS_RENDER_MERMAID=1` (needs `mmdc`).
- Two glossaries cross-reference concepts — including a full **LangGraph → this repo → Azure** map.

A small echo of "repository as system of record": if an agent (or teammate) can't see it in-context, it doesn't exist.

Status & how to run it

- `7 / 7 lessons` implemented locally & tested.
- Suite is deterministic and cloud-free.
- `Azure wiring pending` — the next increment.

```
# run the full suite (no cloud needed)
uv run --extra dev pytest

# one test
uv run --extra dev pytest \
  tests/test_guardrails.py::test_normal_input_allowed

# the API locally
uv run uvicorn harness.app:app --reload
curl localhost:8000/health
```

Tooling is `uv` (provisions Python ≥3.11). Despite the Go-style path, this project is **Python**.

Takeaways

- **Thin over managed.** The runtime is mostly configuration on Azure primitives, not from-scratch infrastructure.
- **The seam pattern + "state the leak"** let you build and test offline, then swap to Azure without touching logic.
- **The seven patterns are the vocabulary** of production agents: guard, persist, sandbox, remember, route, supervise, ask a human.

Questions?

Design: `docs/architecture.md` · Plan: `docs/azure-implementation-plan.md` · Concepts: `docs/glossary.md`, `docs/langgraph-glossary.md`